

# ragent Project Scaffolding Manual

This guide explains how to use ragent's `/new` command to scaffold a brand-new software project in the current directory. The command creates the directory layout ragent expects (`.ragent/`, `specs/`, `log/`, `.gitignore`, `AGENTS.md`), generates a minimal, runnable “Hello, world” application for the selected language and application type, generates starter documentation (`README.md`, `QUICKSTART.md`, `STATS.md`, `docs/`), initialises a git repository with an initial commit, and optionally creates a remote repository on GitHub or GitLab and pushes.

The same functionality is available as the `ragent new` CLI subcommand (and as `ragent run "/new ..."`), with identical flags and identical behaviour.

**Scope:** the `/new` slash command, the `ragent new` CLI surface, flag validation, the empty-directory guard, language/type/stack support, remote hosting setup, failure containment, and progress streaming. For spec management commands, see `docs/howtos/spec.md`. For reverse-engineering an existing repository, see `docs/howtos/reverse.md`.

---

## 1. Purpose and capabilities

The `/new` command answers a common moment in a developer's day: **“I have an empty directory and I want a working, agent-friendly project — right now.”**

Instead of manually creating files, running `git init`, setting up hosting, and writing boilerplate docs, `/new` automates the whole sequence:

- **Validates the target directory** — refuses to scaffold anything that is not empty (ragent's own artifacts `.ragent/`, `log/`, `target/` are ignored), so an existing project can never be clobbered.
- **Creates the ragent workspace** — `.ragent/` (config placeholder and custom-agents directory), `specs/`, `log/` (gitignored), a pre-populated `.gitignore`, and an `AGENTS.md` project-guidelines stub.
- **Generates a runnable hello-world artifact set** for the selected language and application type — buildable with the language's default toolchain and nothing else. Data and build formats scaffold sample documents instead of runnable programs.
- **Layers an optional framework stack** on top of the base layout (e.g. an axum HTTP route starter for Rust).
- **Generates starter documentation** — `README.md` (title, description, build/run instructions), `QUICKSTART.md` (prerequisites, exact run command, expected first output), `STATS.md` (timestamp, language, type, stack, file count, ragent version), and a `docs/` folder.
- **Initialises git** — `git init` plus an initial `Initial scaffold` commit on the current branch.
- **Optionally creates hosting** — with `--github` or `--gitlab` the command creates a private repository via the hosting API, sets it as `origin`, and pushes the initial commit.

### What it does not do

- It does not import an existing project — “new” means greenfield only.
- It does not overwrite existing files — if a target file already exists it is reported in the summary and left untouched.
- It does not pick your language for you — `--language` and `--type` are required.
- It does not push to remotes other than the freshly created `origin`, nor branches other than the current branch the initial commit lands on.

---

## 2. Quick start

Open a terminal in an **empty** directory and run ragent there:

```
/new --language rust --type cmdline
```

The TUI streams the scaffold steps into the message window as they happen:

```
Scaffolding 'my-project'...
[ .. ] guard passed, emitting files
[ ok ] guard passed, emitting files
[ .. ] writing project files
[ ok ] writing project files (14 created)
[ .. ] initialising git repository
[ ok ] initialising git repository
[ .. ] no hosting flag, no remote
[ ok ] no hosting flag, no remote
[ ok ] scaffold complete
```

When the run finishes, the progress panel is followed by the summary message:

```
From: /new
Scaffolded my-project (rust, cmdline)
Target: /home/you/projects/my-project
Created 14 file(s):
+ .gitignore
+ AGENTS.md
+ Cargo.toml
+ src/main.rs
...
Git: initialised new repository, initial commit created
Remote: none
```

To scaffold straight onto GitHub:

```
/new --language rust --type cmdline --github
```

This additionally creates a **private** GitHub repository named after the directory, sets it as origin, and pushes the initial commit.

---

## 3. Command syntax

**3.1** `/new --language <lang> --type <type> [flags]`

Scaffold a new project in the current working directory.

### Flags

| Flag                                                    | Required | Values                                                                                                                                                                                                                                                                                                                                                                                                      | Default when omitted                       |
|---------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>--languageyes</code><br><code>&lt;lang&gt;</code> |          | any value from<br><code>/new help:</code><br>46 canonical<br>languages<br>spanning the<br>codeindex<br>scanner set<br>(application<br>languages<br>such as rust,<br>python, go,<br>typescript,<br>shell; data<br>and build<br>formats such<br>as json, yaml,<br>sql, cmake,<br>maven);<br>dialect aliases<br>ts/tsx,<br>js/jsx, c++,<br>c_header/cpp_header,<br>sh/bash, yaml,<br>sv, vhd, tf,<br>scad, kts | none — validation error                    |
| <code>--type</code><br><code>&lt;type&gt;</code>        | yes      | library,<br>cmdline, tui,<br>gui                                                                                                                                                                                                                                                                                                                                                                            | none — validation error                    |
| <code>--stack</code><br><code>&lt;name&gt;</code>       | no       | known stacks<br>for the<br>language,<br>e.g. axum,<br>warp, raylib,<br>gtk4 (Rust)                                                                                                                                                                                                                                                                                                                          | no stack layer applied                     |
| <code>--github</code>                                   | no       | flag                                                                                                                                                                                                                                                                                                                                                                                                        | no remote is created and nothing is pushed |
| <code>--gitlab</code>                                   | no       | flag                                                                                                                                                                                                                                                                                                                                                                                                        | no remote is created and nothing is pushed |

`--github` and `--gitlab` are **mutually exclusive**: supplying both aborts with a conflict message and zero file mutations.

### 3.2 `/new help`

Bare `/new` and `/new help` print the detailed help page (FR-018): the command purpose, a usage line, per-argument documentation — each argument’s name, one-line description, whether it is required or optional, its accepted values, and the default behaviour when it is omitted — and two worked example invocations (one minimal without hosting, one with a hosting flag). The accepted `--language` and `--type` value lists and the known-stack examples are derived from the scaffolder’s internal registries (the language registry

and the `STACK_RECIPES` table), so the help text cannot drift from what the command actually accepts. The `ragent new help` CLI surface prints the same page with binary-form usage lines.

### 3.3 Input rules

- Language values are matched case-insensitively; scanner dialect ids are accepted as aliases (`ts/tsx` for `typescript`, `js/jsx` for `javascript`, `c_header/cpp_header`, `sh/bash` for `shell`, `yml, sv` for `verilog`, `vhd, tf` for `terraform`, `scad` for `openscad`, `kts` for `gradle_kts`).
- Stack values are matched case-insensitively and scoped to the selected language — `--stack axum` applies to Rust projects; on Python it warns and falls back to the base layout.
- An unknown stack **warns and continues** with the base layout; the warning appears in the summary, not as a failure.
- The project name is taken from the current directory name. Renaming the directory before scaffolding renames the project.

## 4. What the command generates

### 4.1 ragent workspace (always)

| Path                                  | Purpose                                                                                                                           |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>.ragent/config.json</code>      | Valid-JSON configuration placeholder (the live loader reads <code>.ragent/ragent.json</code> ; this placeholder is informational) |
| <code>.ragent/agents/README.md</code> | Placeholder for custom agent definitions (OASF JSON or Markdown)                                                                  |
| <code>specs/README.md</code>          | Placeholder for project specifications (manage with <code>/spec</code> )                                                          |
| <code>log/</code>                     | Runtime logs (created empty, gitignored)                                                                                          |
| <code>.gitignore</code>               | Language-specific build-artifact rules plus <code>.ragent/</code> , <code>log/</code> , <code>target/</code>                      |
| <code>AGENTS.md</code>                | Minimal project-guidelines stub with the language's run/test commands                                                             |

### 4.2 Application artifacts (per language/type)

The 26 application languages (`rust`, `python`, `go`, `typescript`, `javascript`, `c`, `cpp`, `java`, `kotlin`, `ruby`, `swift`, `csharp`, `lua`, `zig`, `nim`, `elixir`, `erlang`, `haskell`, `ocaml`, `r`, `dart`, `php`, `perl`, `shell`, `zsh`, `fish`) scaffold a full four-app-type hello-world set. The original first-release four:

| Language            | Manifest                    | cmdline/tui/gui entry    | library entry                             | Run                          | Test                                      |
|---------------------|-----------------------------|--------------------------|-------------------------------------------|------------------------------|-------------------------------------------|
| <code>rust</code>   | <code>Cargo.toml</code>     | <code>src/main.rs</code> | <code>src/lib.rs</code>                   | <code>cargo run</code>       | <code>cargo test</code>                   |
| <code>python</code> | <code>pyproject.toml</code> | <code>main.py</code>     | <code>src/&lt;name&gt;/__init__.py</code> | <code>python3 main.py</code> | <code>python3 -m unittest discover</code> |

| Language   | Manifest     | cmdline/tui/gui entry | library entry | Run                       | Test        |
|------------|--------------|-----------------------|---------------|---------------------------|-------------|
| go         | go.mod       | main.go               | <name>.go     | go<br>run                 | go<br>test  |
| typescript | package.json | src/main.ts           | src/index.ts  | npx<br>tsx<br>src/main.ts | npm<br>test |

The tui and gui app types reuse the cmdline entry path with a terminal-UI or GUI starter body for the language (for Rust, a minimal starter with the framework integration point marked by a comment).

The 20 data/markup/build formats (toml, yaml, json, xml, html, css, scss, sql, markdown, protobuf, verilog, vhd, terraform, openscad, cmake, gradle, gradle\_kts, maven, nix, hcl) scaffold a manifest plus cmdline and library sample documents that carry a Hello, world! greeting; --type tui/--type gui degrade to a manifest-only layout (no sample source is planned for those types). Every language id the codeindex scanner understands therefore has a scaffold. Where a canonical manifest exists it is used verbatim (pom.xml for maven, build.gradle/build.gradle.kts for Gradle, CMakeLists.txt for cmake, main.tf for terraform, index.html for html, NOTES.md for markdown); the remaining stubs use a <language>-notes.txt manifest. Data formats report (none - data format) as their run/test commands.

/new help prints the full accepted-value list, derived from the same registry the validator uses (NFR-001).

### 4.3 Stack layering (--stack)

Known stacks append a dependency line to the language manifest and wrap the entry point with a framework-specific starter snippet. First-release stacks are Rust-only:

| Stack          | Dependency added                         | Starter provides                                                               |
|----------------|------------------------------------------|--------------------------------------------------------------------------------|
| axum           | axum = "0.7" + tokio macros/rt           | Router with a / route handler and<br>axum::serve                               |
| warp           | warp = "0.3" + tokio macros/rt           | warp::Filter hello service and<br>warp::serve                                  |
| raylib<br>gtk4 | raylib = "5"<br>gtk4 = "0.9"             | raylib::init() window loop<br>GTK 4 Application with a Hello,<br>world! window |
| ratatui        | ratatui = "0.29" + crossterm =<br>"0.28" | Ratatui Terminal + Paragraph draw<br>loop with a keypress exit                 |

### 4.4 Documentation scaffold (always)

| Path      | Content                                                                                                                |
|-----------|------------------------------------------------------------------------------------------------------------------------|
| README.md | Project title (directory name), one-paragraph description, entry point / run / test instructions, project-layout table |

| Path           | Content                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------|
| QUICKSTART.md  | Prerequisites (toolchain pointer), the exact run command, expected first output (Hello, world!), next steps |
| STATS.md       | Generated timestamp (UTC), language, app type, stack (or none), generated-file count, ragent version        |
| docs/README.md | Placeholder that materialises the docs/ folder                                                              |

All build/run instructions come from the same per-language recipe data that produces the code artifacts, so the documentation always matches the generated project.

## 5. The empty-directory guard

Before any file is written, `/new` inspects the current directory and refuses to scaffold when it contains anything beyond the ragent artifact allowlist (`.ragent/`, `log/`, `target/`). The refusal reports the offending entries and writes nothing:

From: `/new`

```
[err] directory is not empty; refusing to scaffold. Offending entries: README.md, src/
```

This guard is what makes `/new` safe to type in the wrong window: a non-empty directory can never be partially scaffolded into. Directory entries in the offending list are suffixed with `/` to distinguish them from files.

## 6. Remote hosting (`--github` / `--gitlab`)

### 6.1 GitHub

`--github` uses the same authentication chain as the `/github` tool family: the `GITHUB_TOKEN` environment variable first, then the `~/.ragent/github_token` file written by `/github login`. Without a token the remote step fails with an auth message and the local scaffold is kept.

The flow: `GET /user` (resolve login) -> `POST /user/repos` (create a private repository named after the directory) -> `git remote add origin` -> `git push -u origin <current-branch>`. If a repository with that name already exists (HTTP 422), the existing repository is reused as `origin` instead of failing.

### 6.2 GitLab

`--gitlab` reads `GITLAB_TOKEN` (env) or `~/.ragent/gitlab_token`, and the instance base URL from `GITLAB_URL` (env) or `~/.ragent/gitlab_config.json` (`instance_url`), defaulting to `https://gitlab.com`. The flow hits `GET /api/v4/user` and `POST /api/v4/projects` with `PRIVATE-TOKEN` auth; a path-taken validation error reuses the existing project as `origin`.

### 6.3 Failure containment

Remote failures never undo the local scaffold. When a remote step fails the summary reports the failed step and an actionable message, and the local repository stays intact:

Remote: failed at github auth: GitHub rejected the token (401); run ``/github login`` to re-authent

Step names are `github auth`, `github repo create`, `gitlab auth`, `gitlab repo create`, `git remote add`, and `git push`.

Re-running the remote flow later (or adding the remote manually with `git remote add + git push`) picks up where the failed run stopped: a local repository without an origin is not an error on retry. The FR-015 idempotent-retry guarantee also covers git itself: when the local repository already exists, the git step reuses it instead of failing on “already initialised”.

---

## 7. Progress streaming and the summary

In the TUI the scaffold runs as a **foreground** operation — no sub-agent, no LLM turn. The blocking file-emission, git, and remote steps run on a worker thread while the UI keeps painting; each step streams a progress line into an in-place-updated progress message (`[ . . ]` while running, `[ ok ]` or `[fail]` when done). The final FR-011 summary lands as the usual `From: /new` message listing: created layout, generated files, chosen language/type/stack, git outcome, and remote status (none / remote URL / failed at step).

Existing files are never silently overwritten: any file that already existed when the emitter reached it is reported in the summary and left untouched.

---

## 8. CLI equivalents

The same surface is available without the TUI:

*# Basic scaffolding*

```
ragent new --language rust --type cmdline
```

*# With a stack layer*

```
ragent new --language rust --type cmdline --stack axum
```

*# With hosting*

```
ragent new --language rust --type cmdline --github
```

```
ragent new --language python --type library --gitlab
```

*# The one-shot prompt surface also dispatches /new*

```
ragent run "/new --language go --type cmdline"
```

*# Usage page*

```
ragent new help
```

`ragent new --github --gitlab` is rejected at the argument-parsing layer. The CLI pipeline performs the same guard `-> emit -> git -> remote` sequence in the foreground and prints the same summary to stdout.

---

## 9. End-to-end examples

### Example 1: Minimal Rust command-line project

```
/new --language rust --type cmdline
```

Generates a cargo project (Cargo.toml, src/main.rs printing Hello, world!), the ragent workspace, documentation, git init + initial commit, no remote.

### Example 2: Python library on GitLab

```
/new --language python --type library --gitlab
```

Generates pyproject.toml + src/<name>/\_\_init\_\_.py with a public function, the ragent workspace, documentation, git init, then creates a private GitLab project and pushes. Requires GITLAB\_TOKEN (or ~/.ragent/gitlab\_token).

### Example 3: Rust web service with axum

```
/new --language rust --type cmdline --stack axum
```

The base src/main.rs is replaced by an axum server starter with a / route handler; Cargo.toml gains the axum and tokio dependencies.

### Example 4: TypeScript command-line tool on GitHub

```
/new --language ts --type cmdline --github
```

Generates package.json + src/main.ts, the workspace and docs, git init, and a private GitHub repository named after the directory with the initial commit pushed.

### Example 5: Scaffold, then plan with the spec system

```
# Step 1: scaffold the project
```

```
/new --language rust --type cmdline
```

```
# Step 2: add a tracked spec for the first feature
```

```
/spec create first-feature
```

```
# Step 3: generate the implementation plan
```

```
/spec plan first-feature "tokio + clap based CLI"
```

A freshly scaffolded project is immediately ragent-friendly: AGENTS.md loads as project guidelines, specs/ is the spec home, and .ragent/ is ready for custom agents.

---

## 10. Troubleshooting



| Symptom                                                                | Cause                                                                         | Fix                                                                                           |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| directory is not empty: ...                                            | The guard found non-ragent entries in the cwd                                 | Run <code>/new</code> in a truly empty directory (or a fresh <code>mkdir</code> )             |
| <code>[err]</code> + usage page after <code>/new</code>                | Missing or invalid <code>--language</code> / <code>--type</code>              | Supply both flags with registry values (see the lists in the usage page)                      |
| <code>--github</code> and <code>--gitlab</code> are mutually exclusive | Both hosting flags supplied                                                   | Pick one                                                                                      |
| Remote: failed at github auth: ...                                     | No/malformed <code>GITHUB_TOKEN</code> or <code>~/.ragent/github_token</code> | Export the token or run <code>/github login</code> , then retry the remote flow               |
| Remote: failed at gitlab auth: ...                                     | No/malformed <code>GITLAB_TOKEN</code> or <code>~/.ragent/gitlab_token</code> | Export the token or run <code>/gitlab setup</code> , then retry                               |
| Remote: failed at github repo create: ...                              | Hosting API rejected creation (network, permissions)                          | Check connectivity and token scopes; the existing-repo reuse path only triggers on name-taken |
| Unknown stack warning in the summary                                   | <code>--stack</code> name not in the language's stack registry                | Check spelling; stacks are scoped per language                                                |
| Progress panel shows <code>[fail]</code> writing project files         | Emission error (permissions, disk)                                            | Fix the environment; earlier creations remain on disk and are reported                        |

## 11. Workflow integration

`/new` is the entry point of the greenfield pipeline; `/reverse` is its counterpart for existing repositories:

```
# Greenfield: scaffold, then specify, then implement
/new --language rust --type cmdline
/spec create my-feature
/spec plan my-feature "tokio based"
/spec tasks my-feature
```

```
# Brownfield: reverse-engineer, then specify
/reverse owner/repo --create my-clone
```

The scaffolded `AGENTS.md`, `specs/`, and `.ragent/` layout are the same conventions ragent itself follows, so agents opening the new project pick it up without configuration.